

Unit Flow Converter

Hecho por: Juan Carlos Cadilla Gomez

Objetivo del Proyecto

El propósito de este proyecto es crear una aplicación sencilla capaz de convertir diferentes tipos de unidades de forma rápida y clara. Además del desarrollo técnico, la actividad busca que el alumno aprenda a gestionar un proyecto utilizando Git, trabajando con ramas, realizando merges y subiendo el código a un repositorio remoto en GitHub. Finalmente, el proyecto se publica mediante GitHub Pages para que pueda visualizarse desde cualquier navegador.

Tecnologías Utilizadas

El proyecto está construido utilizando HTML para la estructura de la página, CSS para los estilos y el diseño visual, y JavaScript para la lógica de conversión entre unidades. Para el control de versiones se ha utilizado Git, y GitHub ha servido como repositorio remoto y plataforma de publicación del proyecto.

Funcionalidades

La aplicación permite realizar conversiones entre Celsius y Fahrenheit, entre kilómetros y millas, y también incluye la conversión entre metros y centímetros, que fue añadida en una rama independiente llamada feature-update. Esta última funcionalidad demuestra el uso de ramas para implementar mejoras sin afectar la versión principal del proyecto.

Estructura del Proyecto

El proyecto está organizado en una estructura sencilla y clara. El archivo `index.html` contiene la interfaz principal. La carpeta `styles` incluye el archivo `main.css` con los estilos. La carpeta `scripts` contiene el archivo `converter.js` donde se encuentra la lógica de las conversiones. Finalmente, el archivo `README.md` documenta el propósito y funcionamiento del proyecto.

Flujo de Trabajo con Git

El desarrollo del proyecto siguió un flujo de trabajo profesional. Primero se inicializó el repositorio con `git init` y se realizó un commit inicial en la rama `master`. Después se creó la rama `feature-update` para añadir nuevas funcionalidades sin modificar la rama principal. Una vez completada la mejora, se realizó un commit y posteriormente un merge hacia `master`. Finalmente, el repositorio se conectó con GitHub y se subió el proyecto completo.

```
PS C:\Users\jccad\OneDrive\Escritorio\Unitflow-converter> git init
>>
Reinitialized existing Git repository in C:/Users/jccad/OneDrive/Escritorio/Unitflow-converter/.git/
PS C:\Users\jccad\OneDrive\Escritorio\Unitflow-converter> █
```

Comandos Principales Usados

Durante el desarrollo se utilizaron comandos esenciales de Git como `git init` para iniciar el repositorio, `git add` y `git commit` para registrar cambios, `git branch` y `git checkout` para gestionar ramas, `git merge` para unir las y `git push` para subir el proyecto al repositorio remoto. Estos comandos forman la base del control de versiones moderno.

```
PS C:\Users\jccad\OneDrive\Escritorio\Unitflow-converter> git status
>>
On branch master
Your branch is up to date with 'origin/master'.

nothing to commit, working tree clean
PS C:\Users\jccad\OneDrive\Escritorio\Unitflow-converter> |
```

Añadir archivos al staging

En esta etapa se añadieron los archivos al área de preparación utilizando git add. Después de este paso, git status mostraba los archivos en color verde, indicando que estaban listos para ser confirmados en el primer commit.

```
PS C:\Users\jccad\OneDrive\Escritorio\Unitflow-converter> git add .  
>> git status  
>>  
On branch master  
Your branch is up to date with 'origin/master'.  
  
nothing to commit, working tree clean  
PS C:\Users\jccad\OneDrive\Escritorio\Unitflow-converter>
```


Primer commit

Una vez preparados los archivos, se realizó el primer commit del proyecto utilizando git commit. Este commit inicial registró la primera versión funcional del proyecto y marcó el inicio del historial de cambios.

```
PS C:\Users\jccad\OneDrive\Escritorio\Unitflow-converter> git commit -m "Initial commit"
>>
On branch master
Your branch is up to date with 'origin/master'.

nothing to commit, working tree clean
PS C:\Users\jccad\OneDrive\Escritorio\Unitflow-converter> █
```

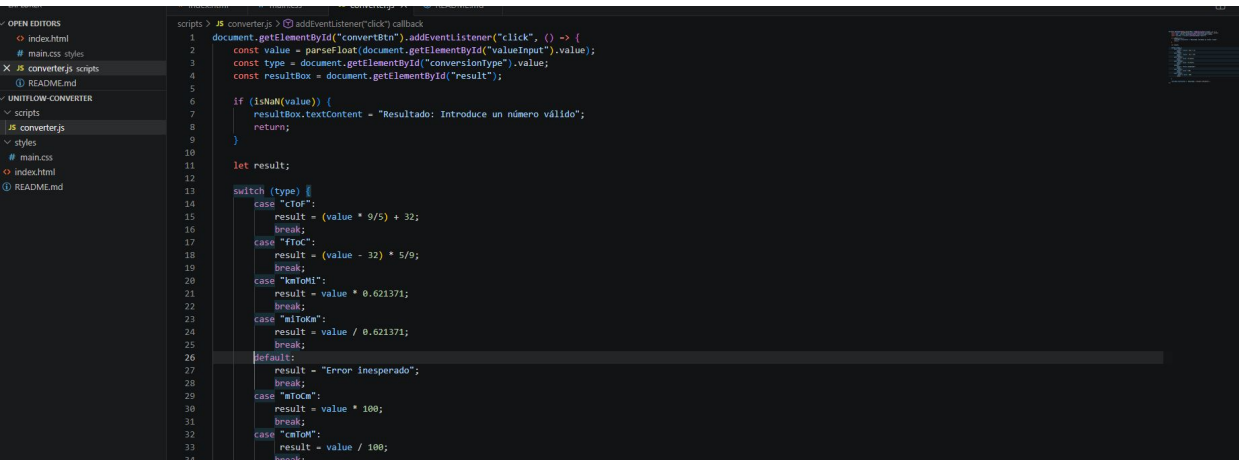
Creación de la rama feature-update

Para añadir una nueva funcionalidad sin afectar la rama principal, se creó una rama llamada feature-update. En esta rama se trabajó de forma aislada, siguiendo buenas prácticas de desarrollo colaborativo y control de versiones.

```
PS C:\Users\jccad\OneDrive\Escritorio\Unitflow-converter> git branch feature-update
>> git checkout feature-update
>>
fatal: a branch named 'feature-update' already exists
Switched to branch 'feature-update'
PS C:\Users\jccad\OneDrive\Escritorio\Unitflow-converter>
```

Modificación del código

En esta fase del proyecto se realizó la modificación del código dentro de la rama feature-update. Se añadió una nueva funcionalidad al programa: la conversión entre metros y centímetros. Esta mejora se implementó directamente en el archivo JavaScript del proyecto, como se muestra en la captura, donde se agregaron los casos mToCm y cmToM dentro del switch principal.



```
scripts > .\ converter.js > addEventListener("click") callback
1 document.getElementById("convertBtn").addEventListener("click", () => {
2   const value = parseFloat(document.getElementById("valueInput").value);
3   const type = document.getElementById("conversionType").value;
4   const resultBox = document.getElementById("result");
5
6   if (isNaN(value)) {
7     resultBox.textContent = "Resultado: Introduce un número válido";
8     return;
9   }
10
11   let result;
12
13   switch (type) {
14     case "ctoF":
15       result = (value * 9/5) + 32;
16       break;
17     case "fToC":
18       result = (value - 32) * 5/9;
19       break;
20     case "mToM":
21       result = value * 0.621371;
22       break;
23     case "MToM":
24       result = value / 0.621371;
25       break;
26     default:
27       result = "Error inesperado";
28       break;
29     case "mToCm":
30       result = value * 100;
31       break;
32     case "cmToM":
33       result = value / 100;
```

Commit de la mejora

Después de implementar la nueva funcionalidad, se añadieron los cambios al área de preparación y se realizó un commit descriptivo. Este commit registró la mejora realizada en la rama feature-update, dejando constancia clara de la modificación en el historial del proyecto.

```
>> git commit -m "feat: added meters to centimeters conversion"
>>
On branch feature-update
nothing to commit, working tree clean
PS C:\Users\jccad\OneDrive\Escritorio\Unitflow-converter> |
```

Merge a master

Una vez completada la mejora, se integraron los cambios de la rama feature-update en la rama master mediante un merge. Este proceso unificó ambas versiones del proyecto, incorporando la nueva funcionalidad a la versión principal.

```
PS C:\Users\jccad\OneDrive\Escritorio\Unitflow-converter> git checkout master
>> git merge feature-update
>>
Switched to branch 'master'
Your branch is up to date with 'origin/master'.
Already up to date.
PS C:\Users\jccad\OneDrive\Escritorio\Unitflow-converter>
```

Conexión con GitHub

En esta etapa se enlazó el repositorio local con un repositorio remoto en GitHub utilizando git remote. Este paso permitió preparar el proyecto para ser subido a la plataforma y mantener un respaldo en la nube.

```
PS C:\Users\jccad\OneDrive\Escritorio\Unitflow-converter> git remote -v
>>
origin https://github.com/jcg0068/unitflow-converter.git (fetch)
origin https://github.com/jcg0068/unitflow-converter.git (push)
PS C:\Users\jccad\OneDrive\Escritorio\Unitflow-converter>
```

Push a GitHub

Finalmente, se subió el proyecto al repositorio remoto mediante git push. Este paso permitió almacenar el historial completo del proyecto en GitHub y habilitó la posibilidad de activar GitHub Pages para su publicación.

```
PS C:\Users\jccad\OneDrive\Escritorio\Unitflow-converter> git push -u origin master
>>
branch 'master' set up to track 'origin/master'.
Everything up-to-date
PS C:\Users\jccad\OneDrive\Escritorio\Unitflow-converter> |
```

GitHub Pages

El proyecto fue publicado utilizando GitHub Pages, lo que permite acceder a la aplicación desde cualquier navegador sin necesidad de instalar nada. La URL final del proyecto es:

<https://jcg0068.github.io/unitflow-converter/>

GitHub Pages

GitHub Pages is designed to host your personal, organization, or project pages from a GitHub repository.

Your site is live at <https://jcg0068.github.io/unitflow-converter/>

Last deployed by  jcg0068 1 minute ago

[Visit site](#)

[Unpublish site](#)

Build and deployment

Source

[Deploy from a branch](#)

Branch

Your GitHub Pages site is currently being built from the `master` branch. [Learn more about configuring the publishing source for your site.](#)

 `master`

 `/ (root)`

[Save](#)

Learn how to [add a Jekyll theme](#) to your site.

Your site was last deployed to the [github-pages](#) environment by the [pages build and deployment](#) workflow.

[Learn more about deploying to GitHub Pages using custom workflows](#)

Custom domain

Custom domains allow you to serve your site from a domain other than `jcg0068.github.io`. [Learn more about configuring custom domains.](#)

[Save](#)

[Remove](#)

Enforce HTTPS

— Required for your site because you are using the default domain (`jcg0068.github.io`)

HTTPS provides a layer of encryption that prevents others from snooping on or tampering with traffic to your site.

When HTTPS is enforced, your site will only be served over HTTPS. [Learn more about securing your GitHub Pages site with HTTPS.](#)

Conclusión

El proyecto Unit Flow Converter demuestra el uso correcto de Git y GitHub en un entorno real. Se ha trabajado con ramas, commits, merges y despliegue web, cumpliendo todos los objetivos de la práctica. El resultado es una aplicación funcional, bien organizada y publicada correctamente en GitHub Pages.